Smart Dine Development Guide

This guide helps developers understand the codebase structure and contribute to Smart Dine.

Project Structure

```
TMS/
├── backend/
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py           # FastAPI app entry point
│   │   ├── database.py       # Database connection & session
│   │   ├── models.py         # SQLAlchemy ORM models
│   │   ├── auth.py           # JWT authentication utilities
│   │   ├── ws_manager.py     # WebSocket connection management
│   │   ├── scheduler.py      # Background task scheduling
│   │   ├── schemas.py        # Pydantic schemas for validation
│   │   ├── routers/          # API endpoint modules
│   │   │   ├── __init__.py
│   │   │   ├── auth.py
│   │   │   ├── branches.py
│   │   │   ├── billing.py
│   │   │   ├── branches.py
│   │   │   ├── dine_auth.py
│   │   │   ├── employees.py
│   │   │   ├── feedback.py
│   │   │   ├── floor_tables.py
│   │   │   ├── insights.py
│   │   │   ├── inventory.py
│   │   │   ├── kitchen.py
│   │   │   ├── menu.py
│   │   │   ├── orders.py
│   │   │   ├── public_customer.py
│   │   │   ├── reports.py
│   │   │   ├── suppliers.py
│   │   │   ├── ws_orders.py
│   │   ├── utils/            # Utility functions
│   ├── alembic/              # Database migrations
│   ├── scripts/              # Utility scripts
│   ├── uploads/              # File uploads directory
│   ├── requirements.txt      # Python dependencies
│   ├── run.py               # Development server script
│   └── init_db.py           # Database initialization
├── frontend/
│   ├── public/
│   ├── src/
│   │   ├── api/              # API client
│   │   ├── components/       # Reusable React components
│   │   ├── contexts/         # React contexts for state
│   │   ├── hooks/            # Custom React hooks
│   │   ├── pages/            # Page components
│   │   ├── types/            # TypeScript definitions
│   │   ├── utils/            # Utility functions
│   │   ├── App.tsx           # Main app component
│   │   ├── main.tsx          # React entry point
│   │   └── vite-env.d.ts
│   ├── package.json
│   ├── tsconfig.json
│   ├── vite.config.ts
│   └── tailwind.config.js
├── docs/                     # Documentation
└── README.md
```

Backend Development

Adding New API Endpoints

1. **Create Router Module:**

   ```python
   # app/routers/example.py
   from fastapi import APIRouter, Depends
   from sqlalchemy.orm import Session
   from app.database import get_db
   from app.auth import get_current_user

   router = APIRouter()

   @router.get("/examples")
   def get_examples(db: Session = Depends(get_db), current_user = Depends(get_current_user)):
       # Your logic here
       return {"examples": []}
   ```

2. **Register Router in main.py:**

```
from app.routers import example
app.include_router(example.router, prefix="/api")
```

3. **Add Pydantic Schemas:**

```
# app/schemas.py
from pydantic import BaseModel

class ExampleCreate(BaseModel):
    name: str
    description: str | None = None
```

4. **Add Database Model:**

```
# app/models.py
class Example(Base):
    __tablename__ = "examples"
    id = Column(Integer, primary_key=True)
    name = Column(String(256), nullable=False)
    # ... other fields
```

Database Operations

- **Migrations:** Use Alembic for schema changes

```
cd backend
alembic revision --autogenerate -m "Add example table"
alembic upgrade head
```

- **Querying:** Use SQLAlchemy ORM

```
# Get all items
items = db.query(MenuItem).all()

# Filter and order
orders = db.query(Order).filter(Order.status == "pending").order_by(Order.created_at.desc()).all()

# Relationships
order = db.query(Order).options(joinedload(Order.items)).first()
```

Authentication & Authorization

- **JWT Tokens:** Generated in `auth.py`
- **Role-based Access:** Check user roles in endpoints
- **Dependencies:** Use `Depends(get_current_user)` for protected routes

WebSocket Implementation

```
from app.ws_manager import manager

# Send message to all connections
await manager.broadcast({"event": "order_updated", "order_id": order_id})

# Send to specific connection
await manager.send_personal_message({"type": "notification"}, websocket)
```

Frontend Development

Component Structure

- **Pages:** Full page components in `src/pages/`
- **Components:** Reusable UI components in `src/components/`
- **Hooks:** Custom logic in `src/hooks/`
- **Contexts:** Global state in `src/contexts/`

API Integration

```
// src/api/api.ts
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_BACKEND_URL,
});

// Add auth interceptor
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export default api;
```

State Management

- **Auth Context:** User authentication state
- **Settings Context:** App settings and preferences
- **Theme Context:** Dark/light theme management

Adding New Pages

1. **Create Page Component:**

```tsx
// src/pages/ExamplePage.tsx
import { useState, useEffect } from 'react';
import api from '../api/api';

function ExamplePage() {
  const [data, setData] = useState([]);

  useEffect(() => {
    api.get('/examples').then(response => setData(response.data));
  }, []);

  return (
    <div>
      <h1>Examples</h1>
      {/* Your JSX here */}
    </div>
  );
}

export default ExamplePage;
```

2. **Add Route in App.tsx:**

```tsx
import ExamplePage from './pages/ExamplePage';

// Inside Routes
<Route path="example" element={<ExamplePage />} />
```

3. **Add Navigation:** Update layout or navigation components

Styling

- **Tailwind CSS:** Utility-first CSS framework
- **Responsive Design:** Mobile-first approach
- **Dark Mode:** Supported via theme context

TypeScript Types

```tsx
// src/types/index.ts
export interface User {
  id: string;
  email: string;
  full_name: string;
  role: 'ADMIN' | 'MANAGER' | 'WAITER' | 'KITCHEN' | 'CASHIER';
  branch_id?: number;
}

export interface Order {
  id: number;
  status: string;
  total_amount: number;
  items: OrderItem[];
  // ... other fields
}
```

Testing

Backend Testing

```python
# tests/test_orders.py
import pytest
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_create_order():
    response = client.post("/api/orders", json={
        "table_id": 1,
        "items": [{"menu_item_id": 1, "quantity": 1}]
    })
    assert response.status_code == 201
    assert "id" in response.json()
```

Frontend Testing

```tsx
// src/components/__tests__/Button.test.tsx
import { render, screen } from '@testing-library/react';
import Button from '../Button';

test('renders button with text', () => {
  render(<Button>Click me</Button>);
  expect(screen.getByText('Click me')).toBeInTheDocument();
});
```

Code Quality

Linting & Formatting

Backend:

- Use `black` for code formatting
- `flake8` or `ruff` for linting
- `mypy` for type checking

Frontend:

- ESLint for linting (`npm run lint`)
- Prettier for formatting
- TypeScript for type checking

Pre-commit Hooks

Consider adding pre-commit hooks for:

- Code formatting
- Linting
- Type checking
- Tests

Performance Optimization

Backend

- Use database indexes for frequent queries
- Implement caching for expensive operations
- Use async/await for I/O operations
- Paginate large result sets

Frontend

- Code splitting with dynamic imports
- Image optimization and lazy loading
- Memoization with React.memo and useMemo
- Virtual scrolling for large lists

Security Best Practices

- Input validation with Pydantic
- SQL injection prevention with SQLAlchemy
- XSS prevention in React
- CSRF protection
- Secure headers with CORS
- Password hashing with bcrypt
- JWT token expiration
- Rate limiting

Contributing Workflow

1. Fork the repository
2. Create a feature branch: `git checkout -b feature/new-feature`
3. Make your changes
4. Run tests and linting
5. Commit with descriptive messages
6. Push to your fork
7. Create a pull request

Commit Message Format

```
type(scope): description

[optional body]

[optional footer]
```

Types: `feat`, `fix`, `docs`, `style`, `refactor`, `test`, `chore`

Deployment

See `docs/DEPLOYMENT.md` for production deployment instructions.

Troubleshooting

Common Development Issues

1. **Database Connection Issues:**
    - Check `.env` DATABASE_URL
    - Ensure PostgreSQL is running
    - Verify database exists
2. **CORS Errors:**
    - Check CORS_ORIGINS in backend `.env`
    - Verify frontend VITE_BACKEND_URL
3. **WebSocket Issues:**
    - Check backend WebSocket endpoint
    - Verify client connection URL
4. **Build Errors:**
    - Clear node_modules and reinstall
    - Check TypeScript errors
    - Verify Python dependencies

Debugging

- **Backend:** Use `print()` statements or logging
- **Frontend:** Browser dev tools, React DevTools
- **Database:** Check SQL logs, use database client

For more help, check the API documentation at `/docs` or open an issue.